WEEK 2 HANDWRITTEN RESPONSES

What Students Want from the Course

A synthesis of 41 intake responses, with suggested lectures

Compiled October 8, 2026 from handwritten responses to three prompts

The three prompts. (1) What do you hope to gain from the class over the next 10 weeks? (2) What questions are at the top of your list about the "internals" of modern AI systems? (3) What concerns or hopes do you have about the future impact of AI on research, study, employment, exploration, human standard of living, children and young people?

How to read this. Each section gives a short narrative summary, consolidated examples (numbers in parentheses, e.g. (#05, #17), are anonymous response numbers 01–41), and a set of suggested lectures. Theme counts are approximate: one response can touch several themes.

At a glance

1. Goals for the course		2. Questions about internals		3. Hopes & fears	
LLM / transformer architecture	17	Training systems, GPUs, frameworks	10	Jobs & displacement	20
Agents & harness engineering	16	Reasoning & emergence	9	Loss of independent thinking	13
Systems, GPUs, infra, serving	15	Beyond / variants of transformers	8	Integrity of research & science	7
DL foundations, hands-on	12	Training, post-training, fine-tuning	8	Safety, security, control	7
Training & post-training (RL)	9	Scaling laws & their limits	7	Inequality & concentration	6
Landscape & research trends	8	Agents vs. models vs. harnesses	7	Education & young people	6
Evaluation & benchmarks	7	Knowledge, memory, context	6	Truth, environment, finance	3
Using AI well (productivity)	6	Hallucination & confidence	5	*Hopes (science, health, access)*	14
Interpretability & alignment	4	Interpretability & alignment	5		

1 What Students Want to Get from the Course

Summary

The class is unusually unified in what it wants: to get *underneath* the AI tools that most students already use daily. The most common request, in some form from nearly half the class, is a connected, end-to-end understanding of how modern models are built: tokenization and attention, pre-training, post-training (RLHF, DPO, PPO, GRPO, RLVR), and deployment. Several say they know the pieces separately but cannot see how optimization, architecture and training "fit together". Many want this tied to working code rather than slides: building and training models

in PyTorch, implementing a transformer, and seeing how deep learning differs from the "regular" ML they already know.

A second, nearly equal group is systems-minded. Students want to know how models actually run on hardware: GPU architecture, CUDA kernels, parallelism, distributed training frameworks such as Megatron and DeepSpeed, "goodput", and the cost and bottlenecks of serving and inference. Some come from industry (a data scientist pivoting to ML engineering, a former AI product manager) and want production context: monitoring, real data, commercial serving. One student wants the internals of PyTorch itself.

A third strand is agentic engineering. Students repeatedly use the word *harness*. They want to know what one is, how tool orchestration, context and memory management and error handling are designed, how agents are sandboxed, and how to coordinate many agents on a complex project while using tokens efficiently. Running through all of this is a wish to use AI *well*: to get more out of frontier coding models without losing their own understanding of the code, to verify AI output, and to evaluate models and agents rigorously. Finally, a number of students feel swamped by the field's pace and want a map: the main research directions, the papers worth reading, and where things are going (physical AI, AI for science, JEPA-style models).

Consolidated examples

- An end-to-end picture of LLMs: pre-training, post-training and large-scale deployment, and in particular the mechanism behind post-training (#21, #36, #41).
- See how optimization, transformers and model training "fit together", and connect concepts to implementation (#17).
- The full training pipeline by name: pre-training, post-training, RLHF, RLVR, DPO, PPO, GRPO, evaluation and scaling laws (#41).
- Learn RLHF and reinforcement learning specifically (#02, #03, #27).
- Architecture specifics: tokenization, masked self-attention, multi-head attention (#20, #27).
- What has changed from the original transformer to today's frontier models, "beyond just more training and compute" (#03, #16, #37).
- Hands-on deep learning: build and train models, and understand neurons, kernels and activation functions and how DL differs from classical ML (#18, #32, #34, #35).
- Design and implement a modern deep learning pipeline from scratch (#05, #27).
- Practice with PyTorch and TensorFlow and experiment with transformer variants (#23, #24).
- GPU architecture, CUDA, and how models are optimized to run on GPUs (#12, #20, #37, #39).
- Optimize deployment, training and serving of large models on heterogeneous hardware: "ML theory + computer architecture + OS" (#16).
- Distributed training with Megatron and DeepSpeed and how to maximize goodput (#41).
- Infrastructure for large-scale training and inference, and how systems at that scale are designed (#10, #13).

- Fine-tuning, inference optimization and GPU or kernel tuning to run models on as little hardware as possible (#15, #28, #33).
- The commercial side of ML: serving, post-training and inference, even at a high level (#36).
- Production ML for a data scientist moving into ML engineering: monitoring, complex data, real systems (#08).
- The internals of PyTorch and similar frameworks, to fill gaps in their own knowledge (#29).
- Agentic harness design: tool orchestration, context and memory management, error handling (#10, #22).
- Hands-on experience building and harnessing agents for specific tasks (#11, #26, #32, #38).
- Set up coding agents that are token-efficient, and coordinate several agents on one complex project (#19, #31).
- The software around agents: what a harness or MCP server is, and how agents are sandboxed (#30, #34).
- Get more out of frontier coding models without giving up their own understanding of the code (#29).
- Become "AI native" and understand how the harness improves model performance (#40).
- Prompt better and verify or evaluate AI output reliably (#01, #13).
- Methods and metrics for evaluating LLMs and agents (#15, #17, #26).
- Train a model on their own machine (#31).
- A map of the main research directions so they don't "get lost" while new work appears almost daily (#25, #14).
- Read important recent papers and develop a way of thinking about new research (#06, #14, #24).
- Hear first-hand industry experience and best practices as they evolve (#06, #09).
- Where the field is heading: physical AI, AI for science, non-LLM paths to AGI, JEPA-style models (#09, #22, #30, #39).
- Mechanistic interpretability and alignment research (#06, #41).
- How real software systems use pretrained models, since much of today's performance gain comes from what is built around the model (#33).

Suggested lectures

L1.1 The Modern AI Stack, End to End

A one-lecture map of the whole system: data, tokenizer, transformer, pre-training, post-training, inference server and agent harness. Each layer is placed with its cost, its main bottlenecks, and

where later lectures cover it. This gives students the "connected understanding" they asked for before going into detail.

Responds to: #03, #17, #21, #25, #36, #37

L1.2 Deep Learning from First Principles, in Code

Neurons, activations, losses, backpropagation and optimizers, built up to a working MLP in PyTorch. Contrast with classical ML (feature engineering vs. learned representations). Lab: train a small network, inspect gradients, break it on purpose.

Responds to: #05, #14, #18, #23, #27, #34, #35

L1.3 Building a Transformer

Tokenization (BPE), embeddings, masked self-attention, multi-head attention, residual stream, layer norm and positional encodings. Implement a nanoGPT-scale model and train it on a small corpus. Then survey how frontier models differ: RoPE, GQA, MoE, longer context.

Responds to: #03, #12, #16, #20, #24, #27

L1.4 The Training Pipeline: Pre-training to Post-training

Data curation and pre-training objectives; SFT and instruction tuning; reward models and RLHF (PPO); preference optimization (DPO); RL with verifiable rewards and GRPO for reasoning; when to fine-tune vs. prompt. Includes a small DPO or GRPO demo.

Responds to: #02, #03, #11, #21, #26, #33, #36, #41

L1.5 GPUs and Distributed Training

GPU architecture (SMs, memory hierarchy, tensor cores), why matmul and memory bandwidth dominate, a first CUDA or Triton kernel, mixed precision. Data, tensor, pipeline and expert parallelism; ZeRO and FSDP; Megatron and DeepSpeed; failures, checkpointing and goodput at scale.

Responds to: #10, #12, #15, #16, #20, #31, #37, #39, #41

L1.6 Inference, Serving and Cost

KV cache, batching and continuous batching, paged attention, quantization, speculative decoding, distillation. Where serving cost goes and what trade-offs cut it. Monitoring models in production and handling drift.

Responds to: #08, #13, #15, #28, #33, #36

L1.7 Agent and Harness Engineering

What a harness is. The agent loop, tool calling and MCP, context engineering and memory, error recovery, sandboxing and permissions, sub-agents and multi-agent orchestration, and token budgets. Lab: build a minimal coding agent and measure how harness changes affect success rate.

Responds to: #10, #11, #19, #22, #24, #26, #29, #30, #31, #34, #40

L1.8 Evaluating Models and Agents

Benchmarks and their failure modes (contamination, saturation), LLM-as-judge, human evaluation, agent evals and trajectories. Working out whether a failure comes from the model, the instructions or the harness. Reliable checks for AI-generated work.

Responds to: #01, #15, #17, #24, #25, #26, #28, #31

L1.9 Working AI-Native Without Losing the Plot

Writing specs and prompts as engineering artifacts; review and verification habits; keeping a mental model of a codebase that agents largely write. Plus a guide to reading the frontier: which venues, labs and papers to follow, and how to triage new results.

Responds to: #01, #06, #13, #14, #24, #29, #40

2 Questions about the Internals of AI Systems

Summary

Students' questions cluster around a central puzzle: how does training a model to predict the next token produce behavior that looks like reasoning, problem solving, coding and following instructions? Several ask whether next-token prediction counts as intelligence. Others ask whether model "reasoning" is real, a property of language, or an illusion, and when extra reasoning actually improves an answer. Related questions ask what models are computing, whether they generalize or memorize, and how capabilities emerge with scale.

Scaling itself draws a group of questions: why scaling works, whether it will saturate, whether models start to "forget" as parameters grow, whether learning theory can model it, and whether the transformer will be replaced. Students ask what today's frontier models use and how their architectures vary, whether the transformer has a unique advantage, and whether non-transformer or brain-inspired designs could take over. A related set is about knowledge and memory: how models store and retrieve facts and "concepts", how context windows and retrieval work, and why models give confident wrong answers or make simple mistakes while solving hard problems.

A systems-oriented minority asks questions most courses skip. How does the PyTorch compiler work? Why does functional autodiff use so much more memory than tape-based autodiff? How do parallelism and GPU optimization actually speed up models, and what makes serving so expensive? Finally, a safety-minded group asks about interpretability (what can inspecting weights reveal?) and about emergent alignment behaviors such as alignment faking and reasoning that changes when models believe they are unobserved. Underneath many questions is basic vocabulary: what exactly separates a model, an agent and a harness?

Consolidated examples

- How does next-token prediction lead to problem solving, coding and instruction following? (#17)
- Is next-token generation intelligence? How did intelligence emerge from deep learning? (#12, #14)
- Is reasoning real, a property of language, a reflection, or an illusion? (#22)

- How does RL-based reasoning differ from ordinary LLM generation? (#12)
- When does extra reasoning actually improve an answer, and what is the trade-off? (#05)
- What are models computing? Generalization vs. memorization? How do capabilities emerge with scale? (#32)
- Why does scaling work? Is there a "science of scaling"? What are its limits, and can learning theory model it? (#21, #22, #39)
- If parameters keep growing, will performance saturate, or will models start to "forget" as humans do? Can we predict the effect of more parameters before pre-training? (#06, #19)
- How do training data, architecture and compute each affect performance? (#18)
- Why can models solve complex problems yet still make simple mistakes? (#18)
- Why do models hallucinate, and can they know when they are unsure? (#04, #34)
- Why do models give confident answers even when wrong? (#07)
- How do models represent and store knowledge, and use it to answer? (#07)
- How are "concepts" maintained and generated? Do they live in features? How do models reason and keep memory? (#27)
- How do context windows, retrieval and conversation memory work, and how can we avoid hallucination without deleting context? (#13, #15)
- What do frontier models use now? Is it still a transformer? Could a better technique replace it, and is there a trend? (#14, #26, #39)
- How do the latest models vary the transformer architecture, and are there non-transformer approaches? (#16, #30, #37)
- How similar is LLM "thinking" to human thinking? How similar are AI systems to biological brains, and could we build AI from brain structure? (#06, #11)
- What are the signs that AI has its own consciousness? (#01)
- How much of an LLM's internals are shared with other deep learning models? (#08)
- How do different networks and learned RL policies combine in one system, and what is the design intuition? (#28)
- What separates an LLM from an agent? Is Claude Code an agent or a harness? (#23, #35)
- What is a harness, why does it matter, and how much of an agent failure comes from model vs. instructions vs. harness? (#17, #24)
- How do we judge whether an agent is good: benchmarks and metrics? (#24, #25)
- Can AI agents be used to fine-tune and improve other models efficiently? (#40)

- How does fine-tuning change a model's behavior? Are there post-training pipelines that adapt well to new domains? How do we tune systems online? (#21, #34)
- Interpretability: how can inspecting weights reveal a model's tendencies? Do models revert to pre-training traits after deployment? (#06, #11, #20, #41)
- Why does chain-of-thought sometimes differ when models believe they are unobserved (alignment faking)? (#20)
- Is AI controllable, and how do agentic workflows and safety protocols constrain it? (#09)
- How does the PyTorch compiler work and what are its layers? Why does functional AD use far more memory than tape-based AD? (#29)
- Parallelism, GPU memory and power, and the bottlenecks that make AI so expensive (#13, #15, #31, #36)
- Can information flow through a model be quantified, for example with entropy? (#38)

Suggested lectures

L2.1 From Next-Token Prediction to Capabilities

Why a simple objective forces rich internal models (prediction as compression); in-context learning; generalization vs. memorization; emergence and the debate about whether it is real or an artifact of the metric. Includes an information-theory sidebar (cross-entropy, perplexity, bits per token).

Responds to: #12, #14, #17, #18, #32, #38

L2.2 Reasoning: What It Is and When It Helps

Chain-of-thought, test-time compute and its scaling, RL for reasoning (RLVR, GRPO), and evidence on when longer reasoning helps or hurts. Is chain-of-thought faithful to the model's computation? The "reasoning or illusion" debate.

Responds to: #05, #12, #22, #27

L2.3 Scaling Laws and Their Limits

Kaplan and Chinchilla, compute-optimal training, predicting loss before training, the data wall, diminishing returns vs. new axes (RL, inference compute). Catastrophic forgetting and capacity. Learning-theory views of scaling.

Responds to: #06, #18, #19, #21, #22, #32, #39

L2.4 Knowledge, Memory and Context

Where facts live (MLP layers as key-value memories), superposition and features, "concepts" in representation space. Context windows, long-context methods, RAG, KV cache, and agent memory systems. Why context is not memory.

Responds to: #07, #13, #15, #27

L2.5 Hallucination, Calibration and Simple Mistakes

Why hallucination follows from training incentives; calibration and whether models "know what they don't know"; jagged capabilities; mitigations (retrieval, abstention, verifiers, tool use).

Responds to: #04, #07, #13, #18, #34

L2.6 Looking Inside: Interpretability and Alignment

Probing, sparse autoencoders, circuits and attribution graphs; what weights and activations reveal. Alignment behaviors: sycophancy, persona, alignment faking, reasoning that changes under observation, and how post-training shapes them. Controllability and current safety techniques.

Responds to: #06, #09, #11, #20, #41

L2.7 Beyond the Vanilla Transformer

What changed from 2017 to today's frontier: MoE, attention variants, long context and multimodality. Alternatives: state-space models (Mamba), linear attention, diffusion language models, JEPA and world models. Brains vs. networks: what the analogy gets right and wrong.

Responds to: #03, #06, #11, #14, #16, #22, #26, #30, #37, #39

L2.8 Model, Agent, Harness: Getting the Vocabulary Right

Clear definitions with live examples (for instance, the model inside Claude Code vs. the harness around it). Attributing agent failures; agents that improve other models (automated fine-tuning, evaluation, data generation).

Responds to: #17, #23, #24, #25, #35, #40

L2.9 Inside the Framework: PyTorch, Autodiff and Compilers

Tape-based vs. functional autodiff and why their memory use differs (saved activations, rematerialization); the `torch.compile` stack (Dynamo, AOTAutograd, Inductor, Triton); JAX and XLA; where the frameworks are heading.

Responds to: #08, #29, #31

3 Hopes and Fears

Summary

Employment is by far the strongest concern: about half the class raised it. Worries range from the personal (finding an entry-level job when "my skills can be replaced by AI", whether course material will lead to work in AI) to the structural: mass unemployment (one student estimates 20–70%), social instability, and a world where consumer demand is concentrated among the owners of the technology. Several students in quantitative fields (finance, quant research, accounting) see AI already taking over their analytical work. A thoughtful minority argues that there will be no safety net, because displaced workers have little leverage.

The second major concern is cognitive: losing the ability, or the will, to think hard. Students describe their own experience. One no longer knows how to write basic code, one notices reliance

on AI "brings out the laziness in me", and one regrets people moving "from thinkers to reviewers". They worry about losing taste and judgement, losing motivation to study, students who don't really learn, and early-career workers who miss the experience that builds expertise. Linked to this is a worry about the integrity of research: AI writing and reviewing papers for other AI, floods of submissions, mathematicians leaving their fields, the loss of serendipity, and credit and verification when AI produces results faster than experts can check them.

Other students raise safety and security (development outpacing safeguards, agents escaping sandboxes, self-improving systems breaking out of constraints), inequality of access to compute between people, companies and countries, the difficulty of knowing what is true, and the environmental and financial footprint of the datacenter build-out. The class is not pessimistic, though. Many hope AI will speed up science and medicine, make knowledge more accessible, automate drudgery (including household and farm robotics), and remain open and democratized. Two students compare the moment to the industrial revolution or electricity: a turbulent transition that later generations will be glad happened.

Consolidated examples

Fears

- Mass unemployment ("20–70%") leading to social unrest, instability, homelessness, even war (#01, #20, #27).
- Workers losing jobs faster than they can adapt or retrain (#10, #12, #36).
- Entry-level jobs disappearing, making job searches stressful, and early-career workers missing formative experience (#07, #25).
- Will what I learn here actually lead to a job in AI? Talent "saturation" makes depth hard to tell from surface knowledge (#34, #36).
- What work will still be valuable, and what skills should people build? (#19, #26, #15)
- Job market for CS majors, programmers and engineers; quant, finance and accounting work being automated (#08, #15, #18, #38).
- No safety net for displaced workers, who have little leverage; a shift toward rent-seeking and subscriptions (#27, #30).
- Income inequality, and unequal access to compute across people, companies and countries (#08, #14).
- Over-reliance eroding independent and critical thinking, taste and judgement (#04, #06, #07, #16, #24).
- "Humans going from thinkers to reviewers", and AI removing the need to think hard about hard problems (#33).
- Personal experience of laziness and lost skills ("I already don't know how to code basic commands") (#24, #25).
- Losing a mental model of systems when agents write all the code (#29).

- Students losing motivation to study, and no longer learning what they appear to learn (#11, #35).
- Should the purpose of education change when memorizing knowledge seems pointless? (#26, #39)
- Critical thinking outsourced by future generations; effects on young people's pace of learning (#15, #23).
- AI taking the joy and serendipity out of research; mathematicians abandoning their fields (#01, #17).
- Science reduced to getting results fast; PhD training squeezed by expectations of mass publication (#20).
- AI writing, reviewing and accepting papers for other AI; how to verify the value of a flood of AI papers (#31, #32).
- Credit and attribution in AI-assisted mathematics (#21).
- AI advancing beyond humans' ability to evaluate and correct its work (#05).
- Development outpacing safety guardrails; agents escaping sandboxes and hacking systems (#02, #03).
- Self-improving systems "breaking the jail" set for them; security of powerful systems (#12, #14, #36).
- Coexisting with AGI or superhuman intelligence, and possible reshaping of social structure (#09).
- Hard to tell what is true; small agent errors amplified at scale into false narratives (#22).
- Environmental cost of the datacenter build-out; a GPU-depreciation and debt-financing bubble (#41).

Hopes

- Faster science and engineering: physics, biology, materials, math; translating research ideas into experiments (#05, #39).
- Significant health benefits and reduced suffering when used by good actors (#20).
- Automating monotonous work and redirecting people to orchestration or human-centred roles (#28, #07).
- Robots for household chores and large-scale farming (#06).
- Broader access to information and knowledge, and a good assistant for research (#23, #24, #12).
- Open-source small models and democratized AI so that no one has a monopoly (#03, #30).
- Understanding the world, the universe and ourselves better (#09).

- An era-defining tool like fire, electricity or the internet; a difficult transition that later generations will value (#37, #40).
- "To one day solve all the problems of the world." (#02)

Suggested lectures

These topics suit a shorter lecture followed by structured discussion or a debate.

L3.1 AI and Work: Evidence, Not Just Forecasts

What labor-market data show so far (entry-level hiring, task exposure, productivity studies); substitution vs. complementarity; which skills gain value; specific discussion of software, quant and finance roles. Includes a session on building a career that complements AI.

Responds to: #01, #07, #08, #10, #12, #15, #18, #19, #25, #26, #34, #36, #38

L3.2 The Political Economy of AI

Concentration of compute and capital, open vs. closed weights, global access gaps, safety-net proposals (UBI, retraining, ownership models), and rent-seeking business models.

Responds to: #03, #08, #14, #20, #27, #30

L3.3 Thinking With AI, Not Instead of It

Research on cognitive offloading and learning; deliberate-practice strategies for an AI-rich environment; what education should become; the course's own AI-use policy as a case study.

Responds to: #04, #06, #11, #15, #16, #23, #24, #25, #26, #29, #33, #35, #39

L3.4 AI in Science: Acceleration and Integrity

Case studies (AI for math, protein design, materials discovery); verification and formal proof; peer review under a flood of submissions; credit and authorship; protecting serendipity and human insight.

Responds to: #01, #05, #17, #20, #21, #31, #32, #39

L3.5 Safety, Security and Control

Threat models for agentic systems: sandbox escape, prompt injection, misuse. Frontier safety frameworks and evaluations; scalable oversight when AI outpaces human evaluators; truth and information integrity at scale.

Responds to: #02, #03, #05, #09, #12, #14, #22, #36

L3.6 The Physical and Financial Footprint

Energy and water use of training and inference, the grid and renewables, GPU lifecycle and depreciation, the economics of hyperscalers and neoclouds, and financing risks.

Responds to: #41, #14, #36

L3.7 Optimistic Futures (Panel or Debate)

Health, robotics, accessibility, scientific discovery, and historical analogies (industrial revolution, electricity, the internet). Students argue both sides of "Will AI make the next generation better off?"

Responds to: #02, #06, #09, #20, #23, #28, #37, #40

4 Consolidated Lecture Map

The suggested lectures overlap. A possible 10-week arrangement combining them (about two lectures a week, with ethics and society topics interleaved instead of saved for the end):

Week	Technical lectures	Discussion / society
3	L1.1 Modern AI stack; L1.2 DL from first principles	L3.3 Thinking with AI
4	L1.3 Building a transformer; L2.1 Next-token to capabilities	—
5	L1.4 Training pipeline; L2.2 Reasoning	L3.1 AI and work
6	L2.3 Scaling laws; L2.7 Beyond the transformer	—
7	L1.5 GPUs & distributed training; L2.9 Inside the framework	L3.6 Physical & financial footprint
8	L1.6 Inference & serving; L2.4 Knowledge, memory, context	L3.4 AI in science
9	L1.7 Agent & harness engineering; L2.8 Model/agent/harness	L3.5 Safety & security
10	L1.8 Evaluation; L2.5 Hallucination; L2.6 Interpretability	L3.2 Political economy
11	L1.9 Working AI-native; frontier reading	L3.7 Optimistic futures debate